



wie kann man Fehler im code auffangen damit die seite nicht crashed

Um zu verhindern, dass deine Webseite bei JavaScript-Fehlern komplett „crasht“ (also z. B. weißer Bildschirm oder nicht mehr reagiert), solltest du Fehler auf mehreren Ebenen abfangen: lokal mit `try...catch`, global mit `window.onerror` und bei Promises mit `unhandledrejection`.^{[1] [2] [3]}

1. Lokale Fehlerbehandlung mit `try...catch`

Für Code, bei dem du Fehler erwartest (z. B. API-Calls, JSON-Parsing, Nutzer-Input), packst du den riskanten Code in einen `try`-Block und fängst Fehler im `catch` ab:^{[2] [3]}

```
try {
  const data = JSON.parse(userInput);
  // Weiter mit data arbeiten
} catch (error) {
  console.error('Fehler beim Parsen:', error);
  // Fehlermeldung für Nutzer anzeigen, statt alles abstürzen zu lassen
  showErrorToUser('Ungültige Eingabe.');
```

Wichtig: `try...catch` fängt nur Laufzeitfehler in diesem Block, keine Syntaxfehler im gesamten Script.^[2]

2. Globaler Fehler-Handler (`window.onerror`)

Um auch unerwartete, nicht abgefangene Fehler zu erwischen, richtest du einen globalen Handler ein. Das verhindert nicht jeden Absturz, gibt dir aber Kontrolle darüber, was passiert, und erlaubt dir, eine Fehlerseite oder eine Meldung anzuzeigen statt eines leeren Bildschirms.^{[4] [5] [1]}

```
window.onerror = function (message, source, lineno, colno, error) {
  console.error('Globaler Fehler:', {
    message,
    source,
    line: lineno,
    column: colno,
    stack: error?.stack
  });

  // Optional: Nutzerfreundliche Fehlermeldung anzeigen
  showGlobalErrorPage();
```

```
// true verhindert die Standard-Fehlerrückmeldung des Browsers
return true;
};
```

Alternativ mit `addEventListener`:^[6] ^[4]

```
window.addEventListener('error', (event) => {
  console.error('Unhandled error:', event.message, event.filename, event.lineno, event.colno);
  showGlobalErrorPage();
  event.preventDefault(); // verhindert Default-Verhalten
});
```

3. Promise-Fehler abfangen (`unhandledrejection`)

Viele moderne Fehler kommen aus asynchronem Code (z. B. `fetch`, `async/await`). Diese lösen oft keine `window.onerror`-Events aus, sondern landen als „unhandled promise rejection“. Auch dafür gibt es einen globalen Handler:^[1]

```
window.addEventListener('unhandledrejection', (event) => {
  console.error('Unhandled Promise Rejection:', event.reason);
  showGlobalErrorPage();
  event.preventDefault(); // verhindert Warning im Console
});
```

Trotzdem solltest du Promises auch lokal mit `.catch()` oder `try...catch` um `await` herum behandeln, um gezielt reagieren zu können.^[3] ^[2]

4. Praxis-Tipps, damit die Seite stabil bleibt

- **Kritische Bereiche schützen:** Code, der UI aufbaut oder Daten verarbeitet, sollte immer in `try...catch` oder mit `.catch()` abgesichert sein.^[3] ^[2]
- **Fallback-UI anbieten:** Wenn ein Modul fehlschlägt, zeige eine Alternative („Fehler beim Laden – bitte later neu laden“), statt die ganze Seite leer zu lassen.
- **Fehler loggen, aber nicht verstecken:** Schicke Fehler an ein Logging-Tool (z. B. Sentry, eigene API), aber zeige dem Nutzer eine verständliche Meldung.^[1]
- **Nicht alles unterdrücken:** `return true` oder `event.preventDefault()` nur nutzen, wenn du wirklich eine eigene Fehlerbehandlung machst; sonst wird Debugging sehr schwer.^[5] ^[6]

Minimalbeispiel für eine robuste Seite

```
<script>
function showGlobalErrorPage() {
  document.body.innerHTML = `
    <h1>Entschuldigung, etwas ist schiefgelaufen.</h1>
    <p>Bitte lade die Seite neu oder kontaktiere den Support.</p>
    <button onclick="location.reload()">Neu laden</button>
  `;
}
```

```

}

window.onerror = function (message, source, lineno, colno, error) {
  console.error('Global error:', { message, source, line: lineno, column: colno, s
  showGlobalErrorPage();
  return true;
};

window.addEventListener('unhandledrejection', (event) => {
  console.error('Unhandled rejection:', event.reason);
  showGlobalErrorPage();
  event.preventDefault();
});

// Beispiel für lokalen Fehler-Schutz
async function loadData() {
  try {
    const res = await fetch('/api/data');
    const data = await res.json();
    renderData(data);
  } catch (err) {
    console.error('Fehler beim Laden:', err);
    showLocalError('Daten konnten nicht geladen werden.');
```

Wenn du mir sagst, ob du React, Vue, Vanilla-JS oder ein Framework nutzt, kann ich dir ein passendes, konkretes Muster für dein Setup zeigen.

✱✱

ich programmiere mit python in vs code

In Python (auch in VS Code) verhinderst du Abstürze hauptsächlich mit try/except-Blöcken und optional finally. Damit fängst du Laufzeitfehler ab, zeigst sinnvolle Meldungen und lässt das Programm weiterlaufen. ^[11] ^[12] ^[13]

Grundprinzip: try / except

```

try:
    # riskanter Code
    zahl = int(input("Gib eine Zahl ein: "))
    ergebnis = 10 / zahl
    print("Ergebnis:", ergebnis)
except ValueError:
    print("Bitte eine gültige Zahl eingeben.")
except ZeroDivisionError:
    print("Durch 0 darf nicht geteilt werden.")
```

Ohne try/except würde das Programm bei falscher Eingabe oder Division durch 0 sofort abbrechen. So läuft es stattdessen weiter. ^[12] ^[11]

Mehrere Fehlertypen abfangen

Du kannst mehrere `except`-Blöcke hintereinander schreiben oder mehrere Exception-Typen zusammenfassen: [\[14\]](#) [\[11\]](#)

```
try:
    ...
except (ValueError, ZeroDivisionError) as e:
    print("Fehler:", e)
```

Oder spezifisch:

```
except ValueError:
    ...
except ZeroDivisionError:
    ...
```

else **und** finally

- `else`: wird nur ausgeführt, wenn **kein** Fehler aufgetreten ist.
- `finally`: wird **immer** ausgeführt (z. B. zum Schließen von Dateien). [\[15\]](#) [\[11\]](#)

```
try:
    datei = open("daten.txt", "r", encoding="utf-8")
    inhalt = datei.read()
except FileNotFoundError:
    print("Datei nicht gefunden.")
else:
    print("Datei erfolgreich gelesen.")
finally:
    # Sicherstellen, dass die Datei geschlossen wird
    try:
        datei.close()
    except NameError:
        pass # datei wurde nie eröffnet
```

VS Code-spezifische Tipps

- Im **Terminal** von VS Code siehst du die Tracebacks (Fehlermeldungen). Mit `try/except` verhinderst du, dass das Skript komplett stoppt. [\[12\]](#)
- Für Web-Apps (z. B. Flask/Django) richtest du oft globale Error-Handler ein, aber im normalen Skript reicht gezieltes `try/except` um kritische Stellen.
- Du kannst Fehler auch loggen (z. B. mit `logging`), statt nur zu `printen`:

```
import logging

logging.basicConfig(level=logging.ERROR)

try:
```

```
...
except Exception as e:
    logging.error("Unbekannter Fehler: %s", e, exc_info=True)
    print("Ein unerwarteter Fehler ist aufgetreten. Bitte versuchen Sie es später noch einmal.")
```

Minimalbeispiel für ein robustes Skript

```
def main():
    while True:
        try:
            zahl = int(input("Gib eine Zahl ein (oder 'q' zum Beenden): "))
            print("Quadrat:", zahl ** 2)
        except ValueError:
            print("Ungültige Eingabe. Bitte eine ganze Zahl oder 'q' eingeben.")
        except KeyboardInterrupt:
            print("\nAbbruch durch Nutzer.")
            break

if __name__ == "__main__":
    main()
```

Wenn du sagst, was für ein Projekt du hast (Skript, Flask-App, Datenanalyse, etc.), kann ich dir ein passendes Fehlerbehandlungsmuster für deinen konkreten Fall zeigen. [\[13\]](#) [\[11\]](#)

✱

<q>Für Web-Apps (z. B. Flask/Django) richtest du oft globale Error-Handler ein, aber

In Flask mit SQLAlchemy verhinderst du Probleme nach einem fehlgeschlagenen `db.session.commit()` am saubersten so:

- Du umgibst die kritischen DB-Operationen mit `try/except`.
- Bei jeder Exception machst du `db.session.rollback()`, damit die Session wieder in einen sauberen Zustand kommt.
- Optional baust du dir einen kleinen Context-Manager, um das Muster nicht überall wiederholen zu müssen. [\[21\]](#) [\[22\]](#) [\[23\]](#) [\[24\]](#)

Minimalmuster pro Route / Funktion

```
from flask import Flask, jsonify
from yourapp.models import db, User

@app.route("/create_user", methods=["POST"])
def create_user():
    try:
        user = User(username="max", email="max@example.com")
        db.session.add(user)
        db.session.commit()
        return jsonify({"msg": "User created"}), 201
```

```
except Exception as e:
    db.session.rollback()
    # Loggen (z. B. mit logging oder Sentry)
    # logging.exception("DB error")
    return jsonify({"error": "Datenbankfehler"}), 500
```

Wichtig: Sobald `commit()` eine Exception wirft (z. B. `IntegrityError`, `OperationalError`), ist die Session „kaputt“ für weitere Operationen im selben Request, bis du `rollback()` machst. [\[25\]](#) [\[26\]](#) [\[21\]](#)

Typische Exceptions gezielt abfangen

```
from sqlalchemy.exc import IntegrityError, SQLAlchemyError

try:
    db.session.add(obj)
    db.session.commit()
except IntegrityError:
    db.session.rollback()
    # z. B. 409 Conflict bei Unique-Constraint-Verletzung
    return jsonify({"error": "Eintrag existiert bereits"}), 409
except SQLAlchemyError:
    db.session.rollback()
    # allgemeiner DB-Fehler
    return jsonify({"error": "Interner Datenbankfehler"}), 500
```

Wiederverwendbarer Context-Manager (empfohlen)

Damit musst du `try/except/rollback` nicht in jeder Funktion neu schreiben. [\[23\]](#) [\[24\]](#)

```
from contextlib import contextmanager
from flask_sqlalchemy import SQLAlchemy as BaseSQLAlchemy

class SQLAlchemy(BaseSQLAlchemy):
    @contextmanager
    def auto_commit(self):
        try:
            yield
            self.session.commit()
        except Exception:
            self.session.rollback()
            raise # damit der Fehler weiter nach oben geht

db = SQLAlchemy()
```

Verwendung:

```
@app.route("/create_user", methods=["POST"])
def create_user():
    try:
        with db.auto_commit():
```

```

        user = User(username="max", email="max@example.com")
        db.session.add(user)
        return jsonify({"msg": "User created"}), 201
    except IntegrityError:
        return jsonify({"error": "Eintrag existiert bereits"}), 409
    except Exception:
        return jsonify({"error": "Interner Fehler"}), 500

```

Globaler Error-Handler für DB-Fehler (optional)

Falls du viele Wege hast, wo DB-Fehler auftreten können, kannst du zusätzlich einen globalen Handler für bestimmte SQLAlchemy-Exceptions definieren und dort immer `rollback()` machen. ^[26]

```

from sqlalchemy.exc import SQLAlchemyError
from flask import jsonify

@app.errorhandler(SQLAlchemyError)
def handle_sqlalchemy_error(error):
    db.session.rollback()
    # logging.exception(error)
    return jsonify({"error": "Datenbankfehler"}), 500

```

Das ersetzt aber nicht das lokale `rollback()` in deinen eigenen `try/except`-Blöcken, wenn du dort spezielle Logik brauchst; es ist eine zusätzliche Absicherung. ^[26]

Kurzfassung

- Um jeden möglichen Fehler bei `db.session.commit()` sicher zu behandeln:
 - `try` um alle DB-Operationen + `commit()`
 - `except` → `db.session.rollback()` + sinnvolle Antwort / Logging
 - Optional: `@contextmanager (auto_commit)` + globaler `@app.errorhandler(SQLAlchemyError)` für zusätzliche Robustheit. ^{[22] [24] [21] [23] [26]}

Wenn du magst, zeig mir kurz deinen aktuellen Code (Route oder Service-Funktion), dann schreibe ich dir eine konkrete, angepasste Version mit `try/except/rollback`.

✱

1. <https://www.tutorialspoint.com/article/how-to-write-a-global-error-handler-in-javascript>
2. <https://www.mediaevent.de/javascript/try-catch-throw-exceptions.html>
3. <https://caddy.tech/docs/de/javascript/try-catch>
4. <https://www.youtube.com/watch?v=ENSrqq8soDU>
5. https://developer.mozilla.org/pt-BR/docs/Web/API/Window/error_event
6. <https://agunechemba.name.ng/2025/09/24/Understanding-Global-error-Event-in-JavaScript-s-Window-Object.html>
7. https://developer.mozilla.org/ru/docs/Web/API/Window/error_event
8. <https://www.youtube.com/watch?v=5HBjCuSaLwY>

9. https://developer.mozilla.org/es/docs/Web/API/Window/error_event
10. <https://danlimerick.wordpress.com/2014/01/18/how-to-catch-javascript-errors-with-window-onerror-even-on-chrome-and-firefox/>
11. <https://www.youtube.com/watch?v=OwkZIOybu10>
12. https://www.youtube.com/watch?v=DAQ4VNGL_Gk
13. <https://www.datanovia.com/de/learn/programming/python/advanced/error-handling.html>
14. <https://www.datacamp.com/de/tutorial/exception-handling-python>
15. <https://informatik-pruefung.de/kurse/python-grundlagen/lektionen/python-fehlerbehandlung/>
16. https://openbook.rheinwerk-verlag.de/python/22_001.html
17. <https://www.youtube.com/watch?v=ICE5njXcup4>
18. <https://www.youtube.com/watch?v=TfojRmxFVBU>
19. <https://www.python-forum.de/viewtopic.php?t=59003>
20. <https://www.datacamp.com/de/tutorial/python-try-except>
21. <https://stackoverflow.com/questions/46205694/how-can-i-better-handle-this-flask-sqlalchemy-commit-rollback/46206875>
22. <https://stackoverflow.com/questions/72847737/sqlalchemy-session-commit-exception-handling-with-rollback>
23. https://blog.csdn.net/qq_37289115/article/details/106352458
24. <https://stackoverflow.com/questions/52232979/sqlalchemy-rollback-when-exception/52234206>
25. <https://www.v2ex.com/t/583388>
26. <https://stackoverflow.com/questions/60230103/flask-sqlalchemy-rollback-best-practices>
27. <https://github.com/miguelgrinberg/flasky/issues/483>
28. <https://www.pythonlore.com/handling-transactions-and-unit-of-work-in-sqlalchemy/>
29. <https://riptutorial.com/sqlalchemy/example/6625/transactions>
30. http://docs.sqlalchemy.org/en/latest/orm/session_basics.html